Fault-Tolerant University Platform

Final Project: Integrated Dependability Engineering Capstone

Course: Fault Tolerance and Dependable Computing

Nariman Sailaubek, Altair Tussupov, Sultan Turan
Group CSE-2502M

November 2026

Abstract

This report presents the design, implementation and evaluation of a dependable university information system (student registration, course catalogue, paid course checkout, assignment submissions, grading and reports). The system runs as separate API, payment-worker and mock-provider services on Kubernetes with MySQL, two Redis instances and S3-compatible storage. We compare a baseline configuration without advanced fault tolerance against an improved (FT) configuration with replicated stateless services, a transactional outbox with reconciliation, bounded retries, timeouts, a circuit breaker, idempotent settlement, cache degradation, scheduled verified backups, alerting, audit logging and login throttling.

Eight experiment groups were run on a single-node OrbStack Kubernetes cluster with the same image for both profiles (midterm/university:local-20261106120520). Under an API instance crash, FT kept 99.4% request availability versus 76.9% for the baseline; under a cache outage 100.0% versus 30.9%. FT recovered 3/3 purchases whose queue message was lost (baseline 0/3), 6/6 worker-crash cases exactly once and 3/3 broker-outage cases with one attempt. A MySQL outage remained user-visible in both profiles (FT 46.4%), which confirms the database as the main single point of failure. All alerts fired within their expected bounds, and 30/30 misuse cases were denied and audited. Validation also found a real defect (the payment relay did not reconnect after a broker restart), which was fixed and re-verified.

Keywords: dependability, fault tolerance, availability, MTTR, transactional outbox, circuit breaker, idempotency, Kubernetes, observability, fault injection

Contents

1. Introduction, motivation and system context
2. Requirements, assumptions and dependability objectives
3. Fault model, threat model and failure analysis
4. Reliability engineering
5. System architecture and resilience design
6. Infrastructure and software fault-tolerance implementation
7. Monitoring, observability and security implementation
8. Verification, validation and experimental methodology
9. Results and baseline-versus-improved comparison
10. Discussion, limitations and future work
11. Conclusion
References
Appendix A. Team contributions
Appendix B. Repository, configuration and raw data

1. Introduction, motivation and system context

A university platform has short periods of very high importance: course registration, payment deadlines and assignment deadlines. Failures at these moments have direct consequences for students: a lost payment, a duplicate charge, a missed submission or an unavailable catalogue. The goal of this project is to engineer such a system so that the most important failures are either masked or contained, and to prove it with measured, repeatable experiments rather than architectural claims.

We continue the university information system from the midterm (Option A). The final project adds measurable dependability objectives, a fault and threat model, alert rules, security controls with misuse tests, a systematic test plan written before the runs, and a consolidated baseline-versus-improved comparison over eight scenario groups.

1.1 Users, workflows and data

Actor	Main functions
Student	Register, sign in (JWT), browse the catalogue, enroll in free courses, buy paid courses, upload one file per assignment, download own files, view grades and purchases
Administrator	Create, edit and archive courses and assignments; grade submissions; export per-course PDF reports
Payment provider (mock)	Authorizes synthetic card payments with six deterministic behaviours (success, transient failures, permanent failure, decline, 3 s delay)
Operator	Deploys, monitors dashboards and alerts, restores backups

The data handled are user accounts (hashed passwords), courses, enrollments, purchases and payment attempts, provider receipts, submissions (private S3 objects with SHA-256), grades and backups. Synthetic card data are validated but never stored, logged or queued.

Critical workflow (multi-step): paid course checkout. (1) The API locks the user and course, validates eligibility and stores the purchase plus an outbox record in one transaction, returning HTTP 201 with a pending purchase. (2) The relay publishes the outbox record to the Redis queue. (3) A worker reserves a durable attempt and lease, then calls the provider with a 2 s deadline. (4) The provider stores a unique receipt before replying. (5) Settlement records the result and creates exactly one enrollment, or schedules a bounded retry. A failure between any two steps must not lose the accepted purchase or charge twice.

2. Requirements, assumptions and dependability objectives

2.1 Critical functions

Function	Criticality	Consequence of failure
Catalogue and read paths	High	Students cannot see or enroll in courses during registration
Checkout and payment processing	Critical	Lost payments, duplicate charges, students not enrolled after paying
Assignment submission and download	High	Missed deadlines, lost or corrupted files
Grades and reports	Medium	Delayed grading; data must never be corrupted
Authentication and authorization	Critical	Data disclosure or unauthorized changes

2.2 Service-level objectives

Each objective is measurable in a defined experiment. Observation windows are the experiment windows defined in the test plan (Section 8), not calendar time.

ID	Objective (FT profile)	Target	Measured in
SLO-1	Read availability during the crash of one API instance	≥ 99% successful probes	E1
SLO-2	Catalogue availability while the cache is unavailable	≥ 99% successful probes	E2
SLO-3	Accepted payments are never lost	0 missing purchases	E2, E4, E6
SLO-4	No duplicate charges or enrollments	0 duplicate receipts, attempts or enrollments	E3, E4
SLO-5	Slow dependencies cannot block callers indefinitely	Each provider attempt < 2.6 s, ≤ 3 attempts	E3
SLO-6	Database outage fails safely and recovers	Only 503/timeout; recovery < 60 s after restart	E2
SLO-7	Traffic spike of 300 req/s	≤ 1% server errors; p95 < 1 s	E5
SLO-8	Incidents are detected	Alert within its bound (40–120 s)	E6
SLO-9	Misuse is denied and audited	100% of cases	E7
SLO-10	Data recoverability	RPO ≤ 15 min (backup period); verified restore	E8

2.3 Assumptions

- One Kubernetes node; host, disk and node failures are outside the tested fault model (documented as single points of failure).
- Faults are injected one at a time; there are no simultaneous independent faults.
- The payment provider is a deterministic mock; transient failures, declines and delays are synthetic fixtures selected by card number.
- Clients may retry checkout with the same idempotency key; the server never trusts client-supplied price, status or attempt numbers.
- Component MTTF values used in the reliability model are assumptions (no field data); MTTR values are measured where an experiment exists.

3. Fault model, threat model and failure analysis

3.1 Terminology

We follow Avižienis et al. (2004): a **fault** is the adjudged cause of an error (for example a crashed process or a lost network message); an **error** is the part of the system state that may lead to a failure (for example a purchase stuck in $\mathrm{processing}$); a **failure** is an event where the delivered service deviates from correct service (for example a student who paid but is not enrolled). Fault tolerance mechanisms aim to stop errors from becoming failures.

3.2 Fault model and failure-impact analysis

#	Fault	Resulting error	Failure without FT	Severity	FT mechanism	Exp.
F1	API process or pod crash	No process serving requests	Catalogue and API unavailable until restart	High	Second replica, readiness-based load balancing, PDB	E1
F2	Cache Redis unavailable	Cache reads fail	Catalogue returns 500	High	Cache bypass to MySQL (graceful degradation)	E2

#	Fault	Resulting error	Failure without FT	Severity	FT mechanism	Exp.
F3	MySQL unavailable	No database connection	Most requests fail	Critical	Fail safe (503), transactions, readiness removal; not masked (SPOF)	E2
F4	Queue Redis down or message lost	Accepted purchase without a queued job	Purchase stuck pending forever	Critical	Transactional outbox, relay, reconciliation	E2, E4
F5	Worker crash mid-payment	Attempt in progress, charge uncertain	Lost or double charge	Critical	Durable attempt + lease, idempotent receipt, stale-result guards	E4
F6	Provider slow or failing	Calls hang or fail	Blocked workers, cascading delays	High	Deadline, bounded retries, circuit breaker	E3
F7	Traffic spike	Queues and latency grow	Timeouts, server overload	Medium	Replicas, Envoy rate limit (admission control)	E5
F8	Data corruption or deletion	Wrong or missing rows	Permanent data loss	Critical	Scheduled backups, verified isolated restore	E8
F9	Misuse or attack	Unauthorized request	Disclosure or tampering	Critical	Authentication, authorization, validation, throttling, audit	E7
F10	Node or host failure	All pods gone	Total outage	Critical	Not masked; GitOps/IaC rebuild only	not tested

3.3 Single points of failure

- One node (host, disk, network): all pods share one failure domain.
- MySQL single instance: all business data and the outbox; measured as user-visible in both profiles (E2).
- Queue Redis and RustFS single instances: accepted work survives a queue outage through the outbox, but uploads need RustFS.
- Backups are stored on the same host as the database (snapshot recovery, not disaster recovery).

3.4 Threat model

Assets: accounts and credentials, grades, payment and enrollment records, private submission files, and service availability. Trust boundaries: the public HTTP gateway (Envoy), the API, internal services (worker, mock provider, datastores) and the operator. Threats are classified with STRIDE (Shostack, 2014).

STRIDE	Threat	Control	Evidence
Spoofing	Forged or tampered JWT; password guessing	RS256-signed JWT (1 h), password hashing, login throttling 5/min per login+IP, 60/min per IP	E7: forged/tampered tokens 401; brute force 429
Tampering	Client sets price, status or admin flag	Server-side DTOs reject unknown fields; price snapshot; DB checks	E7: client price, register as admin, negative price 400
Repudiation	Denied actions leave no trace	Audit events `security.denied` and `security.authentication_failed` with request ID, user, client IP	E7: 100% audited; E6 log correlation
Information disclosure	Reading another student's data or files	Ownership checks return 404; private S3 objects; no card data stored or logged	E7: other student's purchase 404
Denial of service	Request floods, oversized uploads	Envoy local rate limit 200 req/s per proxy; 10,000,000-byte upload limit; timeouts	E5 admission control; E7 oversized upload 400
Elevation of privilege	Student calls admin operations	Role checks on every admin route	E7: 403 and audit record

4. Reliability engineering

4.1 Formulas

Failure rate $\lambda = 1 / \mathrm{MTTF}$ (per hour for the model; per second for experiments: $\lambda = N / T$)

MTBF = MTTF + MTTR (model); operating MTBF = $(T - D) / N$ (experiments)

Availability $A = \mathrm{MTTF} / (\mathrm{MTTF} + \mathrm{MTTR})$

Series (all needed) $A = A_1 \cdot A_2 \cdot \ldots \cdot A_n$ Parallel (k = 2 replicas) $A = 1 - (1 - A_1)^2$

Downtime per year = $(1 - A) \cdot 8760\ \mathrm{h} \cdot 60\ \mathrm{min}$

In the experiments, T is the observation time, D the estimated unavailable time and N the number of contiguous outage episodes. Request availability = successful probes / all probes; sampled time availability = (T − D) / T. A run without failures is right-censored: its MTTF is only known to exceed the window, and MTTR and MTBF are undefined.

4.2 Component model (illustrative, mixed assumed and measured inputs)

The MTTF values below are **assumptions** for illustration; no field failure data exist for this system. The MTTR values are **measured** in this project where an experiment exists (Section 9), and the source of every value is stated. The resulting annual figures are therefore estimates, not observations.

Component	MTTF (h, assumed)	MTTR used	Source of MTTR	λ (1/h)	A	Downtime / year
Node (single host / VM)	4380	1800.00 s	Assumed: reboot + Kubernetes + pod start (not measured)	2.28e-4	0.99988586	59.99 min
Envoy gateway pod	720	2.84 s	Assumed equal to the measured API pod restart	1.39e-3	0.99999890	0.58 min
API pod	720	2.84 s	Measured: baseline api_crash MTTR	1.39e-3	0.99999890	0.58 min
Cache Redis pod	2160	10.01 s	Measured: baseline cache_outage MTTR	4.63e-4	0.99999871	0.68 min
MySQL pod	2160	13.21 s	Measured: baseline mysql_outage MTTR	4.63e-4	0.99999830	0.89 min
Worker pod	720	9.42 s	Measured: FT checkpoint crash recovery	1.39e-3	0.99999637	1.91 min

Read path, baseline (all in series): node · Envoy · API · cache · MySQL.

$$A_{\mathrm{baseline}} = 0.99988\,068 \quad \longrightarrow \quad 62.7\ \mathrm{min\ downtime\ /\ year}$$

Read path, FT: Envoy and API have two replicas each (parallel), and the cache is not in series because the FT profile reads MySQL directly when the cache fails.

$$A_{\mathrm{FT}} = A_{\mathrm{node}} \cdot [1 - (1 - A_{\mathrm{envoy}})^2] \cdot [1 - (1 - A_{\mathrm{api}})^2] \cdot A_{\mathrm{mysql}} = 0.99988\,416 \quad \longrightarrow \quad 60.9\ \mathrm{min\ /\ year}$$

Interpretation. Replication makes the stateless components almost irrelevant (their parallel unavailability is of order 10^{-12} to 10^{-13}), so the FT read path is limited by the single node (assumed 30 min MTTR twice a year, about 60.0 min/year) and by the single MySQL instance. Further replication of stateless services would not improve the estimate; a second node and a replicated database would. This matches the measured result that FT did not improve the MySQL outage (Section 9).

4.3 Observed reliability metrics from the experiments

Values are means over 3 runs per profile and scenario, from `reliability.json` (measured). The observation windows are tens of seconds around a forced fault, so these are experiment-scale rates, not field MTBF.

Scenario	Profile	Request avail.	Time avail.	MTTR (s)	Op. MTBF (s)	MTTF (s)	λ (1/s)
api_crash	Baseline	76.9%	76.7%	2.84	9.37	2.30	0.082
api_crash	FT	99.4%	99.5%	0.20	12.01	2.26	0.027
cache_outage	Baseline	30.9%	29.0%	10.01	4.12	2.24	0.071
cache_outage	FT	100.0%	100.0%	censored	censored	> 13.7 (censored)	0.000
mysql_outage	Baseline	63.5%	30.8%	13.21	5.80	2.24	0.053
mysql_outage	FT	46.4%	21.4%	16.21	4.42	2.26	0.048

5. System architecture and resilience design

The system has three independently deployed application services built from one Symfony codebase: the HTTP API (PHP 8.3 on RoadRunner, also serving the Campus web frontend), the payment worker (Supervisor-managed Messenger consumer, outbox relay and metrics exporter) and the mock payment provider. Stateful dependencies run as separate workloads: MySQL (business data, outbox, receipts), queue Redis (AOF, no eviction), cache Redis (bounded, disposable) and RustFS (private files and backups). Each profile has its own Envoy gateway; monitoring is shared.

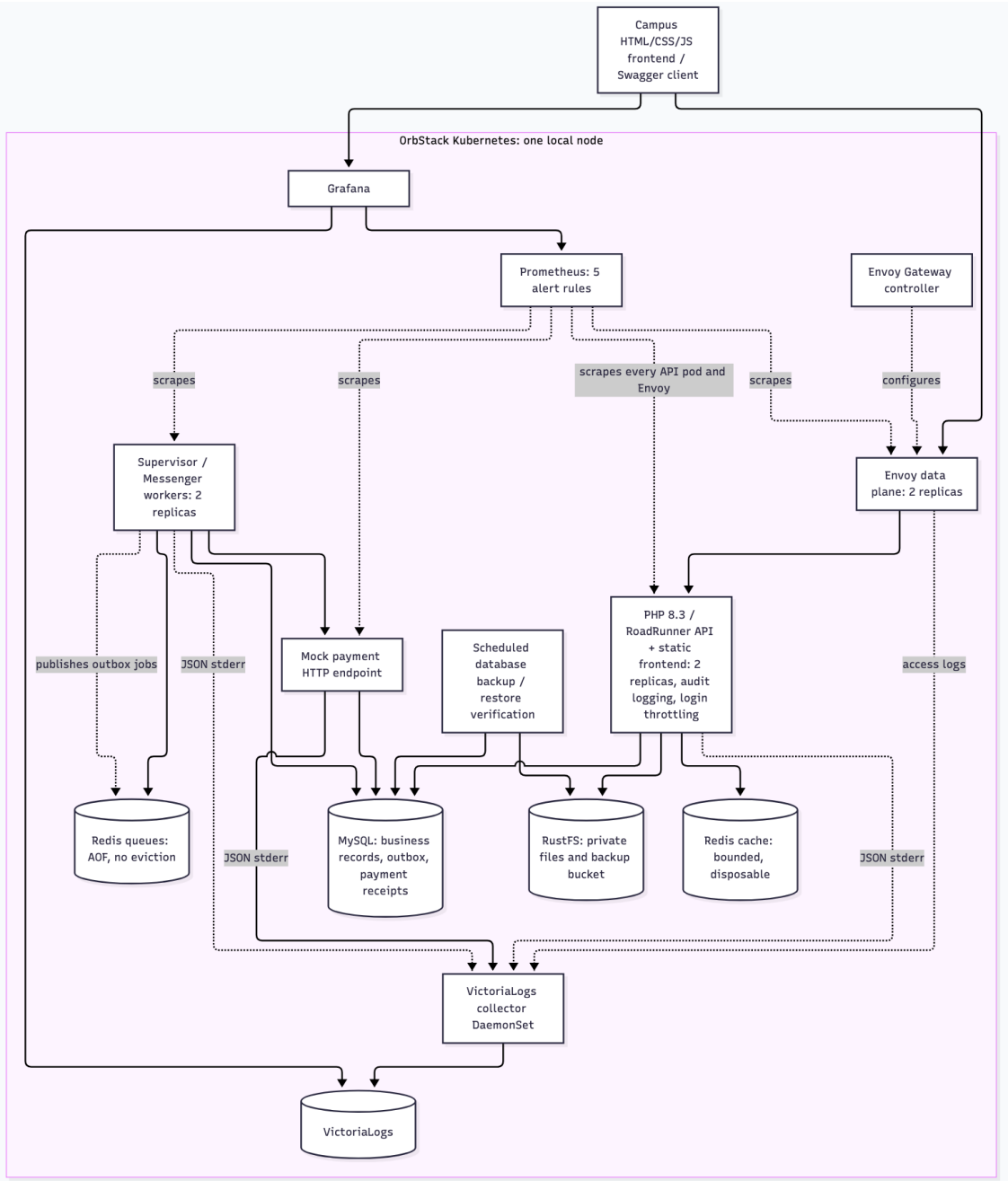


Figure 1. FT architecture on the single-node OrbStack cluster. The baseline uses the same components with one API, worker and Envoy replica and no backups.

5.1 Dependencies and data flows

From → to	Purpose	Failure behaviour (FT)
Client → Envoy → API	All HTTP traffic, 12 s backend / 15 s total timeout, 200 req/s per proxy	Readiness-based routing to healthy pods; 429 above the rate limit
API → MySQL	Authoritative reads and transactional writes	Fail safe with 503; no partial writes
API → cache Redis	Catalogue cache (30 s TTL)	Bypass to MySQL
API → RustFS	Private submission files	Upload compensation deletes orphan objects
Relay → queue Redis	Publish outbox jobs (ID only)	Outbox keeps the intent; relay restarts on broker errors
Worker → mock provider	Payment authorization, 2 s deadline	Bounded retries, circuit breaker

From → to	Purpose	Failure behaviour (FT)
Backup job → MySQL, RustFS	Dump every 15 min to a separate bucket	Verified isolated restore
Prometheus, collector → all	Metrics (5 s) and JSON logs	Alerts on incidents

5.2 Redundancy, replication and backup strategy

Stateless services are replicated (2 API, 2 worker, 2 Envoy pods) with pod anti-affinity preferences and PodDisruptionBudgets. Stateful services are single instances on persistent volumes with durability settings (InnoDB redo log and doublewrite, Redis AOF every second). Durability of accepted work does not depend on Redis: MySQL is the source of truth and the outbox republishes lost jobs. Backups give a recovery point objective of 15 minutes and are verified by restoring into a separate database and comparing ordered row hashes. Multi-node replication of MySQL, Redis and RustFS is documented as future work.

6. Infrastructure and software fault-tolerance implementation

6.1 Infrastructure-level mechanisms

#	Mechanism	Failure addressed	Configuration	Evidence
H1	Multiple instances: 2 API, 2 worker pods	Instance crash	helm/university/values.yaml	E1, E4
H2	Load balancing over ready endpoints only	Routing to failed pods	Envoy HTTPRoute, readiness probes	E1
H3	Redundant gateway: 2 Envoy replicas	Gateway crash; overload	templates/gateway.yaml	E5
H4	PodDisruptionBudget minAvailable: 1	Voluntary disruption	templates/application.yaml	config
H5	Scheduled MySQL backups + verified restore	Data loss or corruption (RPO 15 min)	templates/backup.yaml, app:backup, app:restore-verify	E8
H6	Persistent volumes for all stateful services	Pod replacement	templates/storage.yaml	E2
H7	Durable storage settings (InnoDB, Redis AOF)	Process kill mid-write	templates/storage.yaml	E2, E4
H8	Infrastructure as code (Helm; Argo CD GitOps variant)	Rebuild after total loss	scripts/setup-local.sh, deploy/	reproducible setup

6.2 Software-level mechanisms

#	Mechanism	Implementation	Code / config	Evidence
S1	Bounded retries with backoff	Initial attempt + 2 retries after 2 s and 4 s; only transient errors retry; consumer restarts with 2–30 s backoff	src/Payment/*, docker/consume.sh	E3, smoke
S2	Timeouts	Provider deadline 2 s; Envoy 12 s / 15 s; processing lease 15 s; outbox lease 10 s	framework.yaml, gateway	E3
S3	Circuit breaker	Shared in MySQL; opens after 5 transient failures for 15 s; one half-open probe	src/Payment/CircuitBreaker.php	E3
S4	Idempotency and duplicate detection	Idempotency-Key with request fingerprint; unique keys on enrollment, submission, grade, attempt and receipt; stale-result guards	migrations, SettlePurchaseHandler	E4, smoke
S5	Checkpointing	Durable attempt and lease committed before the provider call; transactional outbox	ClaimPurchaseHandler, OutboxClaimHandler	E4

#	Mechanism	Implementation	Code / config	Evidence
S6	Rollback and recovery	One transaction per handler; rollback on exception; reconciliation of abandoned purchases; upload compensation	Persistence/Transaction.php, PaymentRelayCommand	E4, E8, tests
S7	Health checks	Separate startup, liveness (process) and readiness (dependencies) probes	templates/application.yaml	E1, E2
S8	Graceful degradation	Catalogue served from MySQL when the cache fails	src/Course/CatalogueCache.php	E2
S9	Broker reconnection	Relay exits on a broker error and Supervisor restarts it with a fresh connection (added after validation found the defect)	PaymentRelayCommand, supervisord.conf	E2, E6

6.3 Automated tests

make check-docker runs Composer validation, container and YAML linting, 12 PHPUnit tests with 140 assertions (unit, integration and functional), PHPStan level 5, Doctrine schema validation and OpenAPI export inside the PHP 8.3 image. The tests cover authorization boundaries, enrollment, checkout idempotency, exact upload limits, duplicate submissions, private downloads, grade limits, payment fixtures, durable success replay, an expired-lease race, transaction rollback, brute-force throttling and audit logging. make helm-check lints and renders both charts; make frontend-check runs 6 Node tests for the web frontend.

7. Monitoring, observability and security implementation

7.1 Metrics, logs and correlation

- **Metrics (Prometheus, 5 s):** RoadRunner HTTP metrics (official dashboard, 9 panels), Envoy request counters by response code, and application metrics from the workers: purchases by status, outbox backlog, oldest pending purchase age, payment attempts by status, circuit state and database health.
- **Logs (VictoriaLogs):** structured JSON from Monolog on stderr, collected per pod. Every request logs http.completed with method, route, status and duration. Payment workers log attempts and outcomes; security events are logged at warning level.
- **Correlation:** Envoy assigns an X-Request-ID; the API keeps it, returns it in the response, logs it in every record and stores it in the purchase metadata, so one ID links the gateway, API and payment job logs.

The observability check passed: 9 official RoadRunner panels, live series for 3 key metrics, and 3 correlated payment log records found by request ID.

7.2 Alert rules (incident indicators)

Alert	Expression (simplified)	For	Indicates
DatabaseDown	min by (profile) (university_database_up) == 0	10 s	MySQL unreachable
ApiReplicasDegraded	count(up{component="api",profile="resilient"} == 1) < 2	15 s	Redundancy lost
PaymentBacklogStuck	max by (profile) (university_purchase_oldest_pending_seconds) > 60	10 s	Payments not progressing
PaymentCircuitOpen	max by (profile) (university_circuit_open) == 1	0 s	Provider failing
AuthFailureSpike	rate of Envoy 401/403/429 > 1/s over 1 min	10 s	Brute force or misuse

Rules are evaluated by Prometheus every 5 s and shown in Prometheus and in the Grafana University dashboard (firing alerts and denied-request rate panels). There is no Alertmanager: notifications are out of scope.

7.3 Security controls

1. **Authentication:** stateless RS256 JWT (1 h), password hashing, registration cannot grant admin rights.
2. **Access control:** admin-only routes; students see only their own records (others return 404).
3. **Input validation:** strict DTOs reject unknown fields; server-side price snapshot; database check constraints; exact 10,000,000-byte upload limit.
4. **Login throttling:** 5 attempts/min per login and IP, 60/min per IP, stored in MySQL so all API pods share it; throttled logins return 429 with `Retry-After`.
5. **Rate limiting:** Envoy local limit of 200 req/s per gateway pod.
6. **Audit logging:** `security.denied` for every 401/403/429 and `security.authentication_failed` for failed logins and invalid tokens, without passwords, tokens or bodies.
7. **Data protection:** card data never stored, logged or queued; generated secrets kept out of git; the internal payment endpoint is not routed through the gateway.

8. Verification, validation and experimental methodology

The test plan (`docs/experiments.md`) states the hypothesis, workload, injected fault, observation and pass criterion of every experiment, and was committed to the repository **before** the final runs. Trial runs were used only to debug the scripts; their numbers are not reported.

8.1 Environment

Item	Value
Host	Apple Silicon MacBook, 8 CPU cores, 16 GiB RAM, on power
Cluster	OrbStack Kubernetes v1.36.5, single node, VM limit 8 GiB, local-path volumes
Image (both profiles)	midterm/university:local-20261106120520
Software	PHP 8.3 / RoadRunner 2025.1, Symfony 7.4, API Platform 4.4, Doctrine ORM 3.7, MySQL 8.4.8, Redis 7.4.8, RustFS 1.0.0-alpha.69, Envoy Gateway 1.9.2, Prometheus 3.5.0, Grafana 12.3.1, VictoriaLogs 1.53.0
Probes and load	Python 3.12 on the same host, through each profile's own gateway

8.2 Baseline and improved configurations

Mechanism	Baseline	FT
API / worker / Envoy replicas	1 / 1 / 1	2 / 2 / 2
PodDisruptionBudget	No	Yes
Payment dispatch	Direct after commit	Outbox + relay + reconciliation
Business retries	None (1 attempt)	1 + 2 retries
Circuit breaker	Off	On
Cache failure	Error	Graceful degradation
Scheduled backups	Off	Every 15 min

Mechanism	Baseline	FT
Identical	Image, deadlines, gateway timeouts and rate limit, probes, idempotency, constraints, security controls, alert rules	

8.3 Procedure and measurements

Both profiles run side by side with separate databases; only one receives traffic at a time. `scripts/run-all.sh` executes all scenarios in a fixed order and waits for every pod to be ready between them. Each scenario was repeated 3 times per profile, except the load spike (1 run per profile) and the alert scenarios (1 run per alert). For every run the scripts record UTC start/end, injection time, detection and recovery times, successful and failed requests, latency and an integrity check, and keep the raw samples in `output/final/`. Definitions of detection time, recovery time, availability and the exactly-once check are given in the test plan.

9. Results and baseline-versus-improved comparison

All values are measured on midterm/university:local-20261106120520. Not every acceptance criterion was met; the failed rows are discussed below. The consolidated table (landscape, next page) is generated by scripts/summarize.py; the raw files are listed in the appendix.

9.1 Consolidated results

Exp.	Scenario	Profile	Passed	Success	Avail.	Detect (s)	Recovery (s)	Latency	Recovery actions	Data integrity
1 Instance failure	api_crash	Baseline	–	140/182	76.9%	0.3	2.84	max 149 ms		
1 Instance failure	api_crash	FT	2/3	179/180	99.4%	0.25	0.2	max 95 ms		
2 Dependency: MySQL	mysql_outage	Baseline	–	70/106	66.0%	0.23	13.21	max 3013 ms		
2 Dependency: MySQL	mysql_outage	FT	3/3	57/122	46.7%	0.25	16.21	max 3062 ms		
2 Dependency: cache	cache_outage	Baseline	–	62/200	31.0%	0.23	10.01	max 243 ms		
2 Dependency: cache	cache_outage	FT	3/3	189/189	100.0%		none needed	max 265 ms		
2 Dependency: queue Redis	broker down during checkout	FT	3/3	3/3 accepted and completed			17.39		relay restart + receiver backoff	3/3 with 1 attempt
3 Delay / timeout	circuit breaker	FT	3/3	3/3 good purchase succeeded		8.26	22.14		1 openings/run	3/3 exactly-once; 0 duplicate
3 Delay / timeout	provider 3 s vs 2 s deadline	Baseline	3/3	3/3 terminal			3.77	attempt max 2008 ms	1 attempts/purchase	3/3 no receipt or enrollment
3 Delay / timeout	provider 3 s vs 2 s deadline	FT	3/3	3/3 terminal			35.06	attempt max 2004 ms	3 attempts/purchase	3/3 no receipt or enrollment
4 Interrupted transaction	published message lost	Baseline	–	0/3 recovered			never (censored)			3 purchase(s) stuck pending
4 Interrupted transaction	published message lost	FT	3/3	3/3 recovered			11.47		outbox reconciliation	3/3 exactly-once; 0 duplicate
4 Interrupted transaction	worker killed after_outbox_publish	FT	3/3	3/3 recovered			9.69		lease expiry + replay	3/3 exactly-once; 0 duplicate
4 Interrupted transaction	worker killed after_provider_success	FT	3/3	3/3 recovered			9.14		lease expiry + replay	3/3 exactly-once; 0 duplicate
5 Load spike	300 req/s for 30 s	Baseline	–	6211/9000	69.0%			p50 6 / p95 9 / p99 56 ms	2789 × 429 (admission control)	
5 Load spike	300 req/s for 30 s	FT	1/1	9000/9000	100.0%			p50 6 / p95 150 / p99 1086 ms	0 × 429 (admission control)	
6 Monitoring / alert	ApiReplicasDegraded (api_degraded)	FT	1/1	1/1 fired, 1/1 resolved		30.82	10.27			
6 Monitoring / alert	AuthFailureSpike (auth_spike)	FT	1/1	1/1 fired, 1/1 resolved		33.65	41.02			

Exp.	Scenario	Profile	Passed	Success	Avail.	Detect (s)	Recovery (s)	Latency	Recovery actions	Data integrity
6 Monitoring / alert	DatabaseDown (database_down)	FT	1/1	1/1 fired, 1/1 resolved		12.98	4.11			
6 Monitoring / alert	PaymentBacklogStuck (payment_backlog)	FT	1/1	1/1 fired, 1/1 resolved		75.42	6.17			1/1 exactly-once; 0 duplicate
7 Unauthorized / invalid	10 misuse cases	FT	3/3	30/30 denied as expected						30/30 audited in logs
8 Recovery / rollback	backup + isolated restore	FT	3/3	3/3 checksum OK			1.2			3/3 match source; 0 violations

Table: consolidated results (means over the listed runs). Passed = runs meeting the pre-registered criterion; "–" = no criterion (baseline comparison). Detection and recovery definitions per scenario are in the test plan.

9.2 E1: application instance failure

Force-deleting one API pod made the baseline unavailable until the replacement pod became ready (mean MTTR 2.84 s; 76.9% of probes succeeded). The FT profile served 179/180 probes (99.4%): the second replica and readiness-based routing masked the crash, except one probe (run 1: HTTP 503 0.25 s after the force-delete). Envoy still routed that request to the killed pod before Kubernetes removed its endpoint, and the gateway does not retry because retries are deliberately disabled to avoid replaying non-idempotent requests. One failed probe out of about 60 puts that run below the pre-registered 99% per-run threshold, so SLO-1 is met in 2/3 runs.

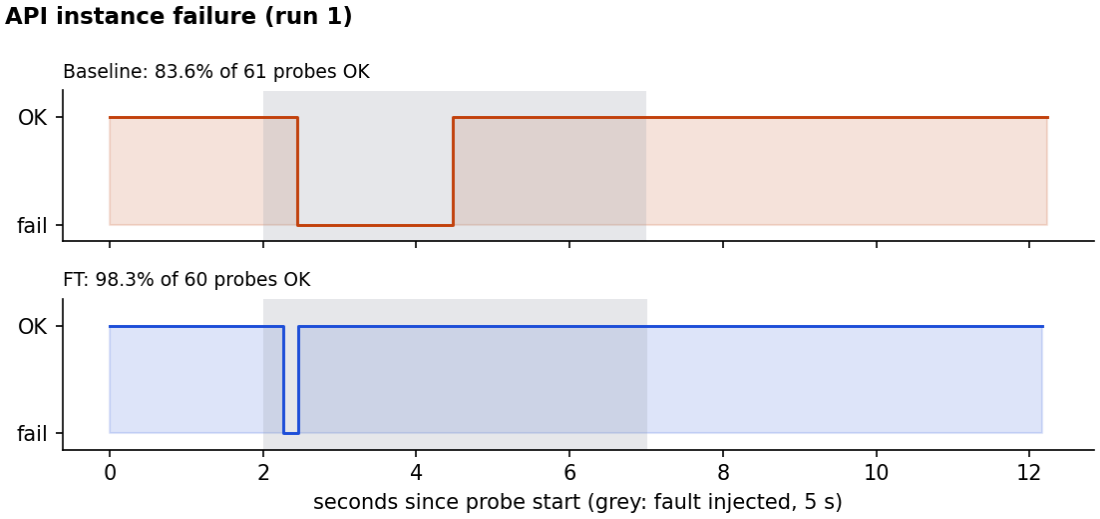


Figure 2. Probe outcome during an API pod crash (run 1).

9.3 E2: dependency failures

Cache. With the cache Redis stopped, the baseline catalogue returned errors (30.9% availability, MTTR 10.01 s) while FT served the catalogue from MySQL with 100.0% availability (SLO-2 3/3).

MySQL. Stopping MySQL caused an outage in both profiles (baseline 63.5%, FT 46.4%; MTTR 13.21 s vs 16.21 s). Every failed probe was a 503 or a client timeout, no partial writes occurred, and service recovered after the restart (SLO-6 3/3). Replicating the API does not help here: MySQL is the single point of failure predicted by the reliability model. FT is not faster to recover because two API pods must both reconnect and pass readiness.

Queue Redis. With the queue Redis stopped during checkout, FT accepted every purchase and completed 3/3 with exactly one attempt, on average 17.4 s after Redis was ready (SLO-3).

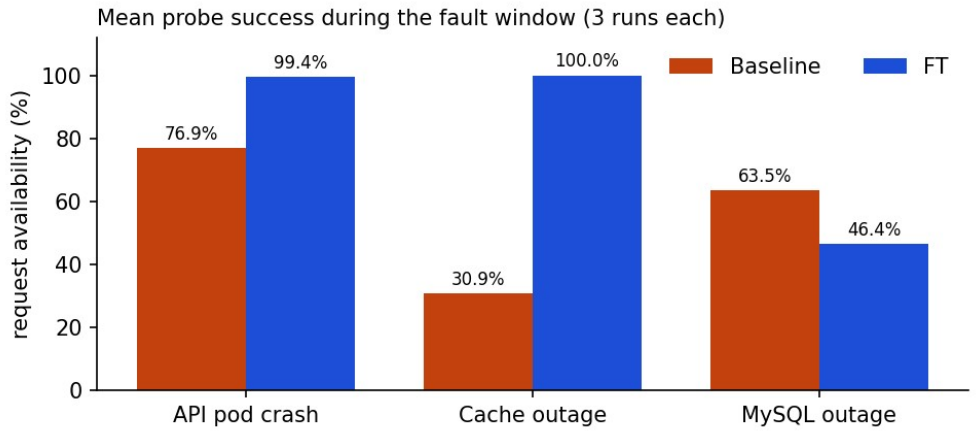


Figure 3. Mean request availability per fault, baseline vs FT.

9.4 E3: network delay, timeout and circuit breaker

With a provider that answers after 3 s and a 2 s deadline, every attempt was cut at about 2008 ms. The baseline gave up after 1 attempt (3.8 s), FT after 3.0 attempts; no timed-out purchase produced a receipt or an enrollment (SLO-4, SLO-5). FT's time to a terminal state grew from run to run (17.0 s, 29.2 s, 59.0 s): the runs were sequential, so their timeouts added up in the shared circuit breaker, which opened and deferred the later retries. The number and length of attempts stayed bounded; only the waiting between them grew. This interaction between retries and the circuit breaker is intended behaviour, but it makes permanent failures slower to report. In the circuit-breaker runs, the circuit opened after 8.3 s on average (1.0 openings per run). A good purchase made while it was open waited without consuming an attempt and then succeeded with exactly one attempt (3/3 runs). Trade-off: failing purchases take longer in FT (retries), and good purchases are delayed while the circuit is open.

9.5 E4: interrupted multi-step transactions

When the published queue message was deleted, the baseline left 3/3 purchases pending until the observation limit (right-censored, never recovered). FT recovered 3/3 through reconciliation in 11.5 s on average. Killing all workers right after the provider charged (before settlement) or right after publishing (before acknowledgement) was recovered in every run, in 9.1 s and 9.7 s on average. Every recovered purchase had exactly one receipt, one attempt and one enrollment: no lost and no duplicate critical records (SLO-3, SLO-4).

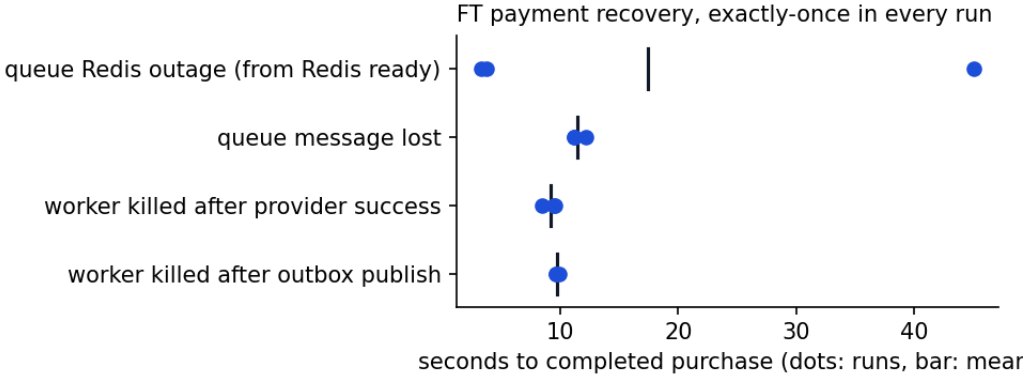


Figure 4. FT payment recovery times per run.

9.6 E5: traffic spike

The spike offered 300 req/s for 30 s between two 20 req/s phases (open loop, fixed rate). Baseline: 207.0 req/s served, 2789 × 429, 0 server/connection errors, p95 9 ms (cool-down p95 22 ms). FT: 300.0 req/s served, 0 × 429, 0 server/connection errors, p95 150 ms (cool-down p95 77 ms). The baseline protects its latency by rejecting about one third of the traffic at its single gateway (admission control). FT serves the full rate through two gateways, at the cost of higher tail latency and a slower return to normal latency after the spike. Neither profile returned a single server or connection error. The load generator stayed on schedule (maximum start delay 5.4 ms), so the offered load is exact.

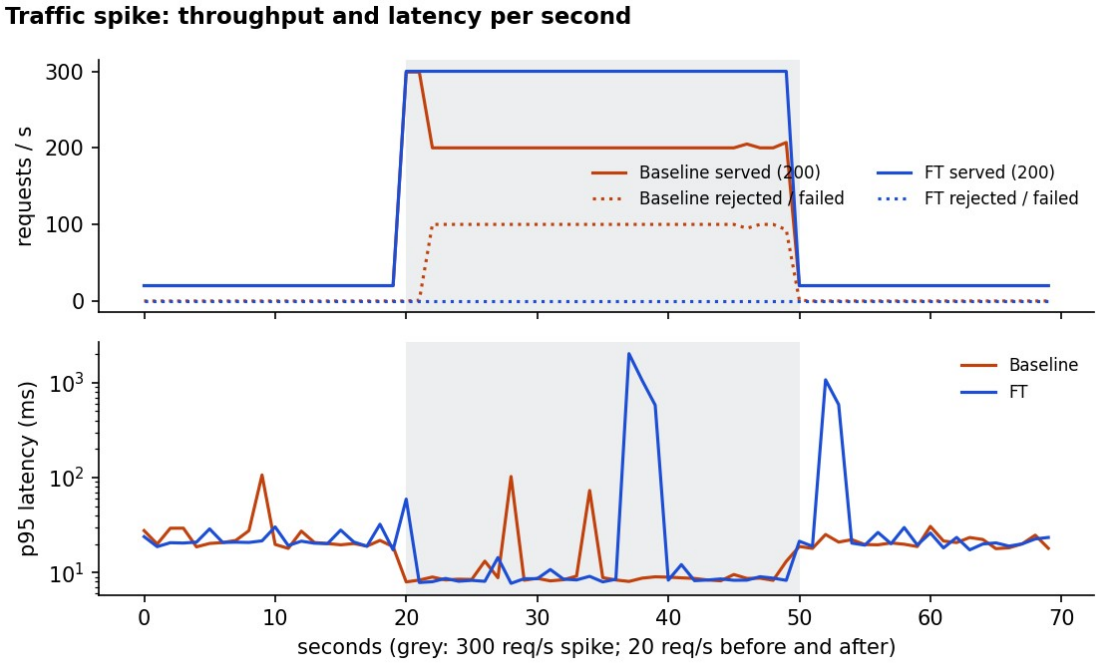


Figure 5. Served and rejected requests and p95 latency per second during the spike.

9.7 E6: monitoring and alert trigger

Alert	Fault injected	Detection (s)	Resolved after repair (s)	Bound (s)
DatabaseDown	MySQL scaled to 0	13.0	4.1	40
ApiReplicasDegraded	API scaled to 1 replica	30.8	10.3	60
AuthFailureSpike	Unauthenticated burst (about 3 req/s)	33.6	41.0	90
PaymentBacklogStuck	Queue Redis scaled to 0 with a pending purchase	75.4	6.2	120

Detection time is dominated by each rule's window and `for` duration plus the 5 s scrape and evaluation interval, which is a deliberate trade-off against false positives: for example, the backlog alert waits for a purchase to be pending for more than 60 s, longer than a normal retry cycle. In every case, the failed requests could be found in VictoriaLogs by their request ID, which is how an operator moves from an alert to the affected requests (SLO-8).

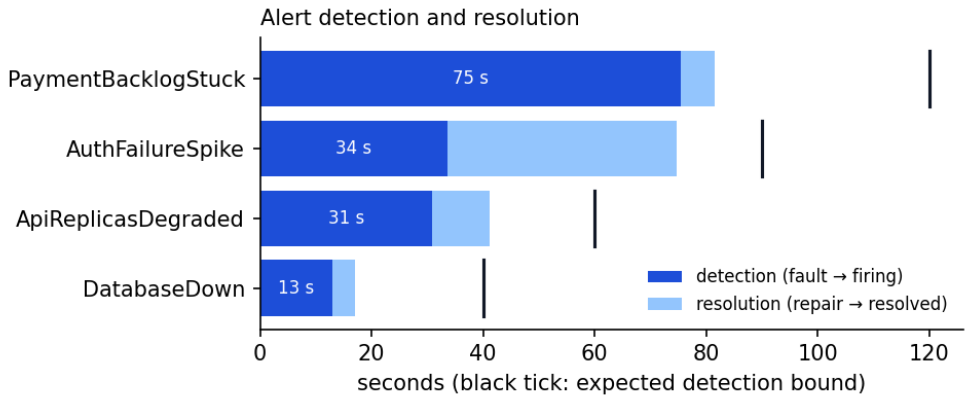


Figure 6. Alert detection and resolution times.

9.8 E7: unauthorized and invalid operations

Across 3 runs, 30/30 misuse cases returned the expected status and 30/30 were found in the audit logs by request ID (SLO-9).

Case	Expected	Observed (all runs)	Audit record
no token	401	401	security.denied
forged jwt	401	401	security.authentication_failed, security.denied
tampered jwt claims	401	401	security.authentication_failed, security.denied
register as admin	400	400	http.problem
student calls admin endpoint	403	403	http.problem, security.denied
read other students purchase	404	404	http.problem
client sets price	400	400	http.problem
admin negative price	400	400	http.problem
oversized upload	400	400	http.problem
brute force login	401,401,401,401,401,429	401,401,401,401,401,429	security.authentication_failed, security.denied

9.9 E8: recovery and rollback

A fresh MySQL dump was taken and restored into a new isolated database 3 times. The backup took 0.26 s and restore with verification 0.94 s on average; the checksum was verified, the restored data matched the live source and there were 0 integrity violations (SLO-10). These timings describe a small dataset and do not predict production restore times.

The functional smoke test passed before the experiment block: all six payment fixtures reached their expected terminal state with the expected number of attempts, idempotent replay returned the same purchase, file checksums and ownership were verified and the PDF report rendered (22224 bytes).

10. Discussion, limitations and future work

10.1 Were the objectives met?

Objective	Result	Met?
SLO-1 instance failure	FT 99.4% vs baseline 76.9%	2/3
SLO-2 cache failure	FT 100.0% vs baseline 30.9%	3/3
SLO-3 no lost payments	queue loss 3/3, broker outage 3/3, worker crashes 6/6 recovered	3/3
SLO-4 no duplicates	12/12 recovered purchases exactly once	yes
SLO-5 bounded calls	max attempt 2008 ms, ≤ 3 attempts	3/3
SLO-6 database outage	fail safe, MTTR 16.21 s; not masked	3/3
SLO-7 traffic spike	p95 150 ms, 0 errors of 9000	1/1
SLO-8 detection	4/4 alerts within bounds	yes
SLO-9 misuse	30/30 denied, 30/30 audited	3/3
SLO-10 recoverability	Verified restore; RPO 15 min by schedule	3/3

10.2 Trade-offs

- **Latency vs availability under load:** FT serves more requests during the spike but with a much higher tail latency; the baseline rejects requests quickly. A production system would add autoscaling and per-route limits.
- **Retries vs time to failure:** bounded retries rescue transient failures but make permanent failures take longer to report (35.1 s on average instead of 3.8 s in E3, more when the circuit breaker is open).
- **Detection speed vs false positives:** alert windows of 10–60 s avoid alerting on single scrapes or normal retry cycles, so detection takes 15–80 s.
- **Resources and complexity:** FT doubles the stateless pods and gateways, and adds an outbox, relay, reconciliation, circuit state and backups, which all need tests and monitoring.

10.3 Negative results and anomalies

- **One failed request during an API crash.** In 1 of 3 FT runs a request reached the killed pod before its endpoint was removed (HTTP 503). A gateway retry for idempotent GET requests on connection failure or 503 would mask this; it was not added after the fact, so the result stands as measured.
- **MySQL outage is not masked.** FT did not improve availability during a MySQL outage; this is consistent with the model and is the most important remaining weakness.
- **Defect found by validation.** During the trial runs, purchases accepted while the queue Redis was down stayed pending forever: the long-running relay kept a dead Redis connection. The midterm had passed this test on an earlier image. The relay now exits on broker errors and Supervisor restarts it (mechanism S9); the final runs confirm the fix. This shows the value of repeating fault-injection tests after every refactoring.
- **Environment anomaly.** In one trial run, right after a load spike, the Kubernetes API was unreachable for about a minute and all application pods restarted together, with no memory or node events. The final procedure waits for all pods between steps; no container restarted during the final runs (all recorded restarts predate them).

10.4 Limitations

- Single node: node, disk and host faults are not tested; replicas share one failure domain.

- The load generator, the probes and both profiles share one machine, so CPU contention can affect latency results.
- Short observation windows and few repetitions (3, or 1 for the spike and alerts) give experiment-scale numbers with wide uncertainty, not field reliability.
- Locally every client has the same IP, so per-IP limits act as one shared limit; local traffic is plain HTTP (TLS is configured in the optional Argo CD deployment).
- The payment provider and card data are synthetic.

10.5 Future work

- Run on at least three nodes with pod anti-affinity and a replicated MySQL (for example InnoDB Cluster or a managed database), Redis Sentinel and distributed object storage.
- Off-host backups and a timed full rebuild drill (RTO for total loss).
- Alertmanager routing and paging; SLO burn-rate alerts.
- Horizontal pod autoscaling and per-route rate limits; longer soak tests with more repetitions.
- Gateway retries for idempotent GET requests on connection failure, to mask in-flight requests to a crashed pod.

11. Conclusion

The project delivers a working, reproducible university platform in two configurations and evaluates them under ten controlled fault, load, monitoring and misuse scenarios with pre-registered criteria. The FT configuration masks stateless failures (a cache outage completely, an API crash except a single in-flight request), never lost or duplicated an accepted payment under message loss, worker crashes or a broker outage, bounds slow dependencies, detects incidents through alerts and correlated logs, denies and audits misuse, and restores verified backups. Not every acceptance criterion was met (see Section 10). The remaining weaknesses are the single database and the single node, which the reliability model and the MySQL experiment both identify as the limiting factors and which define the next steps.

References

Avižienis, A., Laprie, J.-C., Randell, B. and Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. IEEE Transactions on Dependable and Secure Computing, 1(1), 11–33. doi:10.1109/TDSC.2004.2

Beyer, B., Jones, C., Petoff, J. and Murphy, N. R. (eds.) (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.

Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software, 2nd ed. Pragmatic Bookshelf. (Timeouts, circuit breaker, bulkheads.)

Richardson, C. (2018). Microservices Patterns. Manning. (Transactional outbox pattern.)

Shostack, A. (2014). Threat Modeling: Designing for Security. Wiley. (STRIDE.)

OWASP Foundation (2021). OWASP Top 10:2021. https://owasp.org/Top10/

Kubernetes documentation: Pod lifecycle and probes; Disruptions (PodDisruptionBudget). https://kubernetes.io/docs/

Envoy Gateway documentation (v1.9): traffic management, local rate limiting. https://gateway.envoyproxy.io/docs/

Prometheus documentation: alerting rules. https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

Symfony 7.4 documentation: Messenger, Security (login throttling), Rate Limiter. https://symfony.com/doc/7.4/

VictoriaLogs documentation: LogsQL. https://docs.victoriametrics.com/victorialogs/

RoadRunner documentation: metrics and static middleware. https://docs.roadrunner.dev/

Appendix A. Team contributions

Member	Area	Contributions
Nariman Sailaubek	Infrastructure and recovery	Kubernetes/Helm deployment and local cluster setup, experiment scripts for instance and dependency failures, interrupted transactions and backup/restore (E1, E2, E4, E8), reliability model, architecture sections
Sultan Turan	Observability and security	Alert rules and dashboards, audit logging, login throttling and security tests, timeout/circuit, spike, alert and security scenarios (E3, E5, E6, E7), consolidated results tooling
Altair Tussupov	Requirements, verification and report	Requirements and SLOs, fault model, test plan and acceptance criteria, results analysis, report and presentation

Parts of the implementation, scripts and this report were prepared with the assistance of an AI coding assistant (Claude Code). All measurements were produced by running the scripts listed below; no value was entered by hand.

Appendix B. Repository, configuration and raw data

Path	Content
README.md	Setup and reproduction instructions
docs/experiments.md	Test plan, hypotheses and acceptance criteria
scripts/setup-local.sh	Build and deploy both profiles and monitoring to OrbStack
scripts/run-all.sh	Complete experiment procedure
scripts/experiments.py, scripts/scenarios.py, scripts/broker-recovery.py	Fault injection and measurement
scripts/summarize.py, scripts/figures.py	Consolidated table, reliability metrics, figures
helm/university, helm/monitoring	Baseline and FT configuration, alert rules, dashboards
src/, tests/	Application code and automated tests
output/final/	Raw per-run JSON, requests.csv, reliability.csv, results.csv/.md, inventories, run log

Reproduction: make setup-local, then bash scripts/run-all.sh, then node docs/report/build-report.mjs.